From Linear Attention to Gated DeltaNet: A Derivation Forced Out by Constraints

ARC-AGI-3 project side note · V1 · 2026-10-12

0. How this started

This derivation was not planned. Our ARC-AGI-3 agent runs on Qwen3.6-35B-A3B. While tuning vLLM's prefix cache we found that about 3/4 of this model's layers are linear attention (Gated DeltaNet, "GDN" below) and 1/4 are ordinary softmax attention. A linear-attention layer keeps no per-token K/V; it keeps a single fixed-size state matrix S. So the cache can only be reused at positions where a snapshot of S was stored, and any edit to the conversation prefix forces a full recomputation.

To understand what this S really is, we decided — without reading the paper — to start from "remove the softmax" and derive the GDN update rule forward.

Rules of the derivation. Bo does the derivation. Claude only grades: "holds / does not hold", plus where the error is — no hints, no answers. Midway a finer grading format was agreed: two layers — (1) is the step correct *within Bo's own logic*, and (2) is the step on a path that reaches GDN.

The value of this note is Section 2. It keeps the full back-and-forth, including wrong turns, wrong grades and corrections. Anyone can look up the final formula; how the constraints pushed it out step by step is what this derivation actually produced.

1. Starting point: three formulas on the table

Softmax attention

$$o_t = \sum_{i \le t} \frac{\exp(q_t^\top k_i)}{\sum_{j \le t} \exp(q_t^\top k_j)} \, v_i$$

Linear attention (drop the softmax)

$$o_t = \sum_{i \le t} (q_t^\top k_i)\, v_i = \Big(\sum_{i \le t} v_i k_i^\top \Big) q_t, \qquad S_t = \sum_{i \le t} v_i k_i^\top = S_{t-1} + v_t k_t^\top, \qquad o_t = S_t q_t$$

with $S_t \in \mathbb{R}^{d_v \times d_k}$, a fixed size that does not grow with sequence length.

Target form (the GDN update was known before the derivation; the goal was *why* it looks like this)

$$S_t = \alpha_t \, S_{t-1} \big(I - \beta_t \, k_t k_t^\top \big) + \beta_t \, v_t k_t^\top, \qquad o_t = S_t \, q_t$$

2. The full derivation, round by round

In chronological order. “Bo” is the deriver’s own words (translated from Chinese, lightly cleaned); “Grade” is the grade given at the time; “Note” is added afterwards.

Phase 0: before deriving — what softmax actually does

> **Bo**: So softmax’s normalization is implemented by Gated DeltaNet?
>
> **Grade**: Does not hold. GDN does not reimplement softmax normalization; it fixes the problems that appear *after* softmax is removed, by other means.
>
> **Bo**: It’s because e^x amplifies tokens with large inner products — similar ones — and then the normalization keeps the output stable. I can derive GDN myself.
>
> **Grade**: Holds.

Phase 1: locating the problem — what is lost without softmax

Round 1

> **Bo**: GDN is there to fix the missing softmax. $S_t = S_{t-1} + v_t k_t^\top$ just keeps accumulating, and the sign is unclear.
>
> **Grade**: Holds.

Note: $q_t^\top k_i$ can be positive or negative; the weights are neither non-negative nor normalized, and S_t only grows.

Round 2

> **Bo**: Positive and negative mixed, so we need to squash it into 0–1; but there will be an assumption: tokens closer to t matter more.
>
> **Grade**: “Closer matters more” holds — that is the gate decay α_t. “Mixed signs, so squash into 0–1” does not hold.

Round 3 (aside: positional encoding)

> **Bo**: Is the positional encoding added directly to q?
>
> **Grade**: Does not hold. The softmax layers use RoPE: q and k are each rotated by a position-dependent angle — a multiplication. GDN layers use no explicit positional encoding; position comes from the order in which decay acts, plus a very short causal convolution.
>
> **Bo**: So each of q, k gets its positional encoding added onto its own raw vector. What about v — does it use positional encoding? Feels like no, it doesn’t take part in attention scoring.

> **Grade**: “Added onto its own raw vector” does not hold (it is a rotation, not an addition); v gets no positional encoding — holds.
>
> **Bo**: That “add” wasn’t addition — it’s pairs of entries multiplied by a rotation matrix.
>
> **Grade**: Holds.

Round 4

> **Bo**: So GDN recasts plain linear attention as some optimization problem?
>
> **Grade**: Holds.

Round 5

> **Bo**: The problem of linear attention is that softmax is gone. Softmax stabilizes the output and amplifies closely related token pairs. So we need a similar optimization here — what to optimize is the question.
>
> **Grade**: Holds.

Round 6

> **Bo**: So the first assumption, “tokens closer to t matter more”, feels like e^x amplifying closely related pairs?
>
> **Grade**: Does not hold. The problem is the word “close”: e^x amplifies tokens that are close **in content** (large $q^\top k$); decay favors tokens that are close **in position**. Different things.

Round 7

> **Bo**: Are content-similar tokens amplified in linear attention? But dissimilar ones get amplified too…
>
> **Grade**: Holds. In linear attention similarity only weights linearly by $q^\top k$, with no exponential amplification; decay looks only at position, so nearby tokens are lifted whether or not they are similar.
>
> **Bo**: Then it comes down to how q_t relates to the different k_i.
>
> **Grade**: Holds.

Phase 2: a wrong road — starting from the output recursion

Round 8

> **Bo**: First write the recursion between o_t and o_{t-1}.

The grader wrote the recursion but added “what actually recurses is the state S_t, not the output o_t”. That was an out-of-bounds hint; Bo called it out on the spot (“did I ask for a hint?

do what I say"). Rewritten exactly as asked, without S:

$$o_t - o_{t-1} = \sum_{i \le t-1} \left((q_t - q_{t-1})^\top k_i \right) v_i + (q_t^\top k_t)\, v_t$$

Round 9

> **Bo**: We said q_t must be related to every k_i, but S_t breaks that relation — that's why I wanted to look at o_t.
>
> **Grade**: Does not hold; the problem is "breaks".
>
> **Bo**: Hides it, then.
>
> **Grade**: Holds.

Note: $S_t q_t = \sum_i (k_i^\top q_t) v_i$ identically; no pairwise relation is lost — it is folded into S_t.

Round 10

> **Bo**: What about deriving from o_t vs. o_{t-1}?
>
> **Grade**: This direction does not reach GDN.
>
> **Bo**: (pointing at $o_t = o_{t-1} + S_{t-1}(q_t - q_{t-1}) + (k_t^\top q_t) v_t$) So we should look from here?
>
> **Grade**: Does not hold; this is still an o_t recursion.
>
> **Bo**: So we should look at the attention score, not touch o directly.
>
> **Grade**: Does not hold; the problem is "attention score".

Note: Bo then pointed out, "you obviously know I meant looking at S_t." The grader graded the literal words — the grader's mistake.

Phase 3: moving to the state S_t — decay and the "core information"

Round 11

> **Bo**: S_t itself should be some weighted average — like the AdamW feeling.
>
> **Grade**: Holds.
>
> **Bo**: But AdamW's weighted average follows directly from an identity; here it's recast as an optimization problem — a different approach.
>
> **Grade**: Holds.

Round 12

> **Bo**: Decaying S_t is the natural intuition, but S_t must also keep some core information? Optimize for that.
>
> **Grade**: Holds.

> **Bo**: What should that core information be? Guide me, but no direct hint or answer — keep it short.

The grader gave one guiding question: "S is read as $o = Sq$. Suppose the next query q happens to equal the key k_t just written — what do you want $S_t k_t$ to equal?"

Round 13 (a back-and-forth about q, several exchanges, condensed)

> **Bo**: q_t, o_t, k_t are naturally kept; what we decay is S_{t-1}, so the question is what to preserve in $S_{t-1}q_t$?
> **Grade**: Does not hold; the problem is at $S_{t-1}q_t$ … nothing written into S contains q.
> **Bo**: $o_t = S_{t-1}q_t + (k_t^\top q_t)v_t$ — the second term isn't decayed; only the first is.
> **Grade**: Holds.
> **Bo**: Then "kept" was right.
> **Grade**: In the sense "the second term is not decayed", holds; the grader had been grading "what is stored in S". What remains: the "core information" means what S should retain.
> **Bo**: So it is: what must be kept when S_{t-1} decays, since S_t's decay acts on S_{t-1}.
> **Grade**: Holds.

Round 14

> **Bo**: Positions nearer t keep more of S's information: $t-1$ decays by α, $t-2$ a bit more, in some ratio. Then just decay by α^n.
> **Grade**: (first) Does not hold; the problems are "α^n" and "just".
> **Bo**: Don't reverse-engineer. This really is a standard way to decay (e.g. discounted expectations in RL).
> **Grade**: (corrected) As a decay scheme, holds. The grader had graded against the final GDN form, which it should not have done.

Round 15

> **Bo**: The problem here is that α is hard to tune; it may decay too fast and lose too much. So we need some explicit expression of information: S_{t-1} decays, but stays as close as possible to the information that must explicitly be kept — like RL staying close to SFT while still changing.
> **Grade**: Holds.

Note: this round already names both parts of the final structure: decay (α), and "change as little as possible while hitting a target" (a proximal correction).

Phase 4: what is the core information — by elimination

Round 16

> **Bo**: The explicit information is like "S_t is a sum of $vk^\top$ matrices"; q_t is not involved. So maybe it's information kept like $Ax = \lambda x$?
>
> **Grade**: First half holds; second half does not — the problem is "$Ax = \lambda x$".
>
> **Bo**: But in the end it's $S_t q_t$, so decompose S_t and keep the top eigen-directions. That keeps the information of the matrix S itself… it should — S itself *is* the information.
>
> **Grade**: Does not hold; the problems are "keep the top eigen-directions" and "S itself is the information".
>
> **Bo**: So the information expressed by S itself is not what we want; we want to approximate some information in the ordinary attention score.
>
> **Grade**: First half holds; second half does not — the problem is "attention score".
>
> **Bo**: Or rather: the way k and v interact in ordinary attention.
>
> **Grade**: Holds.

Note: this rules out the spectral road. What must be preserved is not S's energy as a matrix, but its behavior as a key→value store.

Round 17

> **Bo**: List a comparison — only the comparison — and I'll compare. It feels right in front of me, just missing some trick.

The grader produced a table whose third row substituted $q = k_j$ and expanded $S_t k_j$. That walked the key step for Bo — the second overstep of the session. After Bo flagged it, the table was cut to its first two rows:

	Softmax attention	**Linear attention**
What is stored	every (k_i, v_i), verbatim	$S_t = \sum_{i \le t} v_i k_i^\top$
Read with q	$o = \sum_i \mathrm{softmax}(q^\top k_i)\, v_i$	$o = S_t q = \sum_i (k_i^\top q)\, v_i$

Phase 5: removing q — replace q by k_j

Round 18

> **Bo**: All I see is $q \approx k_j$: the larger the inner product of q and k_j, the more similar. So it's like replacing q by k_j inside linear attention.
>
> **Grade**: Holds.
>
> **Bo**: But we don't know the specific k_j — so compute it in advance?
>
> **Grade**: No.
>
> **Bo**: Then the formula itself finds the optimal k_j and keeps it through the iterations of S.

> **Grade**: Does not hold; the problem is "finds the optimal k_j".

Round 19 (another back-and-forth about q)

> **Bo**: In effect we use $k_j \approx q_t$ to do some decay of S_t — or, to keep softmax's association between k and v.
>
> **Grade**: Second half holds; first half does not — the problem is "$k_j \approx q_t$".
>
> **Bo**: But that's exactly what was said before: q_t is close to k_j.
>
> **Grade**: "q is close to k_j" holds on its own; what does not hold is attaching it to "doing the decay of S_t" — they belong to different stages.
>
> **Bo**: So q is closest to k_j; in the original attention $q \cdot k_j$ is largest, and that information must be kept — kept when S_{t-1} decays.
>
> **Grade**: Does not hold; the problem is the object being kept.
>
> **Bo**: Lay out my logic — don't reverse-engineer, purely my own chain.

The grader laid out Bo's chain: (1) without softmax, S must decay but keep core information — holds; (2) the kept information involves only k, v — holds; (3) in softmax, when q is closest to k_j the readout is mainly v_j — holds; (4) so when S_{t-1} decays, keep "$q \cdot k_j$ is largest" — graded does-not-hold, as it reintroduces q and contradicts step (2).

> **Bo**: No — that's about the final expression of o. I wasn't wrong.
>
> **Grade**: (corrected) Read as "the final expression of o", step (4) holds: the readout o should keep the behavior "whichever k_j the query q most resembles, read out mainly v_j". The grader had read it as "what is stored in S" — a wrong grade.

Round 20

> **Bo**: So it's "$k_j^\top k_j$ is largest"? That's the way to remove q from S.
>
> **Grade**: Holds.

Note: the turning point of the whole derivation. The test "the query equals some key already written" eliminates the unknown q.

Round 21

> **Bo**: (looking at $S_t = \sum_i v_i k_i^\top$) It feels like the values on S_t's main diagonal.
>
> **Grade**: Does not hold; the problem is "diagonal".
>
> **Bo**: Right — the diagonal isn't $k_j k_j^\top$, it involves v.
>
> **Grade**: Holds.
>
> **Bo**: Suppose we know some k_j; then $S_t k_j$ should approximate the value $k_j k_j$.
>
> **Grade**: Does not hold; the problem is "$k_j k_j$" (read as the inner product $k_j^\top k_j$: one side is a vector, the other a scalar).

> **Bo**: S_t contains no explicit $k_i \cdot k_i$, yet it still has to come out of $S_t k_i$. $S_t \in \mathbb{R}^{d_v \times d_k}$, $k_i \in \mathbb{R}^{d_k}$ — they can be multiplied. Multiply it out, one step.

$$S_t k_j = \sum_{i \le t} v_i \, (k_i^\top k_j)$$

> **Bo**: So $S_t k_j$ is a weighted sum of v_i for $i = 1..t$, weighted by the inner products of k_i and k_j. What we need: when q_t multiplies S_t, the $k_j^\top k_j$ term in $S_t k_j$ should be the largest. Not fully closed yet.
>
> **Grade**: Holds.

Round 22

> **Bo**: So while decaying S_t, if $k_j^\top k_j$ itself is not large enough, scale it up. I'm deriving within my own system.
>
> **Grade**: (first does-not-hold, then corrected) Consistent within Bo's logic — holds; but the road "enlarge $k_j^\top k_j$" does not reach GDN.

Note: from this round on, grading uses two layers: "correct within the deriver's own logic" and "on a path to GDN or not".

Phase 6: explicit vs. implicit

Round 23

> **Bo**: $S_t k_j$ itself is fine; from here to GDN.
>
> **Grade**: Logic holds; starting from $S_t k_j$ does reach GDN.
>
> **Bo**: But the crux: we don't know this k_j, and cannot know in advance which j it is.
>
> **Grade**: Logic holds; but treating that as the blocker does not reach GDN.
>
> **Bo**: When q_t arrives we could compute once and learn j — is that not allowed?
>
> **Grade**: It can be computed, but then every k_i must be kept; the state is no longer fixed-size, which is back to ordinary attention. Does not reach GDN.
>
> **Bo**: That's what I said earlier — it can't be computed explicitly; it's implicit somehow.
>
> **Grade**: Logic holds; this direction reaches GDN.
>
> **Bo**: Isn't implicit just some kind of "finding j"?
>
> **Grade**: Logically one can read it that way; but as long as the idea is "find j" it does not reach GDN — the problem is the word "find".
>
> **Bo**: It has nothing to do with the main diagonal. I feel it just needs to hold for *every* i?
>
> **Grade**: Both hold; this direction reaches GDN.

Note: the second turning point. Do not *find* j; require the property for **every** key ever written, so whichever j a future q matches, it works.

Round 24

> **Bo**: As S_t evolves, whichever k_j, its term is preserved as much as possible, so any q_t that equals some k_j works. The question is how to formalize "assume q_t is this k_j".
>
> **Grade**: Logic holds; reaches GDN.
>
> **Bo**: But keeping every one may not be solvable. As S_t evolves, terms for j far from t still decay. Perhaps that's no coincidence: S_t must decay, otherwise it diverges; but when it decays, the terms for j near t are preserved the most.
>
> **Grade**: Logic holds; reaches GDN.

Phase 7: write it as an optimization problem and solve

Round 25: the shape of the objective

> **Bo**: So: the product of the decayed S_t and k_t should differ minimally from something. That something reflects $k_t k_t$; if every $k_t^\top k_t$ is kept as large as possible, that's exactly the closed-form decay expression for S_t.
>
> **Grade**: "Decayed S times k_t differs minimally from something" — logic holds, reaches GDN. "That something reflects $k_t k_t$" — consistent with Bo's logic, but this part does not reach GDN.
>
> **Bo**: All we have now is k_t, v_t, q_t?
>
> **Grade**: Does not hold; the problem is "all".
>
> **Bo**: I know there's also S_t, S_{t-1}; but what can go directly into the "target being subtracted" is only k_t, v_t, q_t. (Typo $S_t q_t$ corrected to $S_t k_t$.)
>
> **Grade**: Logic holds; reaches GDN.

Round 26: what the target is

> **Bo**: You said "that something reflects $k_t k_t$" is wrong, but the main idea is to keep each k_j term's information so any $q_t = k_j$ works — that's contradictory.
>
> **Grade**: Not contradictory; the main idea is fine; the problem is the words "inner product".
>
> **Bo**: Of course $S_t k_t$ and $k_t^\top k_t$ can't be compared — vector vs. scalar. The thing to match is a vector in $\mathbb{R}^{d_v}$ that still carries the information of $k_t^\top k_t$ — some implicit form of it.
>
> **Grade**: Logic holds; reaches GDN.
>
> **Bo**: The thing to match has nothing to do with q_t; this is now mathematical-optimization modeling.
>
> **Grade**: Both hold.
>
> **Bo**: A bit of a stretch, but could it be something isomorphic to $v_t k_t^\top k_t$?
>
> **Grade**: Logic holds; reaches GDN.
>
> **Bo**: So it's isomorphic to $(v_t k_t^\top) k_t$ but not $(v_t k_t^\top) k_t$ itself?
>
> **Grade**: Does not hold; the problem is "but not".

> **Bo**: Right — it can only be $(v_t k_t^\top)k_t$... feels forced out.
>
> **Grade**: Holds.

Note: looking back at $S_t k_j = \sum_i v_i (k_i^\top k_j)$, the $i = j$ term is exactly $(k_j^\top k_j) v_j = (v_j k_j^\top) k_j$: a vector in $\mathbb{R}^{d_v}$ whose coefficient carries $k_j^\top k_j$. "Keep the whole term, not its coefficient" — that is what was wrong with "inner product" in Round 26.

Round 27: write the optimization problem

> **Bo**: So the objective is $\min \left\| S_t k_t - (v_t k_t^\top) k_t \right\|$?
>
> **Grade**: Holds.
>
> **Bo**: Then $S_t = \alpha S_{t-1} + A$, with A given by this optimization?
>
> **Grade**: Holds.
>
> **Bo**: Substitute S_t, set the derivative to zero, and see what A is?
>
> **Grade**: Holds.
>
> **Bo**: You compute it — this is pure math.

Let $u = (v_t k_t^\top) k_t = (k_t^\top k_t)\, v_t$:

$$\mathcal{L}(A) = \tfrac{1}{2} \left\| (\alpha S_{t-1} + A) k_t - u \right\|^2, \qquad \frac{\partial \mathcal{L}}{\partial A} = \big((\alpha S_{t-1} + A) k_t - u \big) k_t^\top$$

Setting it to zero ($k_t \neq 0$):

$$A\, k_t = (k_t^\top k_t)\, v_t - \alpha S_{t-1} k_t$$

A has $d_v \times d_k$ unknowns but there are only d_v equations, so the solution is not unique.

Round 28: an extra constraint

> **Bo**: And A is a matrix...
>
> **Grade**: Holds.
>
> **Bo**: So there must be another constraint?
>
> **Grade**: Logic holds; reaches GDN.
>
> **Bo**: A should be smaller than the norm of the actual S_t? Something like a spectral-radius kind of measure.
>
> **Grade**: "A should be small" — logic holds, reaches GDN. "Smaller than S_t's norm" and "spectral radius" — consistent with Bo's logic, but these two do not reach GDN.
>
> **Bo**: So we can't solve for A directly; A should be a product of two vectors?
>
> **Grade**: Holds.
>
> **Bo**: $A \in \mathbb{R}^{d_v \times d_k}$, $A =$ vector $\times\, k_t^\top$?
>
> **Grade**: Holds.

Round 29: solve

Substitute $A = x\,k_t^\top$; the left side is $x\,(k_t^\top k_t)$, so

$$x = v_t - \frac{\alpha S_{t-1} k_t}{k_t^\top k_t}, \qquad \boxed{S_t = \alpha S_{t-1}\Big(I - \frac{k_t k_t^\top}{k_t^\top k_t}\Big) + v_t k_t^\top}$$

Check: $S_t k_t = \alpha S_{t-1} k_t - \alpha S_{t-1} k_t + (k_t^\top k_t) v_t = (k_t^\top k_t)\, v_t = u$, the target is hit exactly; for any direction w orthogonal to k_t, $S_t w = \alpha S_{t-1} w$ — only decayed, never rewritten.

> **Bo**: Isn't that it?
>
> **Grade**: Logic holds. Only one difference from the GDN formula remains.

3. Closing: from this solution to the GDN formula

First difference: normalization. GDN L2-normalizes k_t before writing, so $k_t^\top k_t = 1$ and the denominator disappears:

$$S_t^* = \alpha_t S_{t-1}\big(I - k_t k_t^\top\big) + v_t k_t^\top, \qquad S_t^* k_t = v_t$$

Bo's form is more general: it holds without normalizing k; the target is then $(k_t^\top k_t) v_t$.

Second difference: the write strength β_t. Rewriting exactly at every step is too aggressive; with noisy keys it wipes out older memory. GDN does not jump to S_t^*; it interpolates between the decayed old state and S_t^*:

$$S_t = (1-\beta_t)\,\alpha_t S_{t-1} + \beta_t\, S_t^* = \alpha_t S_{t-1}\big(I - \beta_t k_t k_t^\top\big) + \beta_t\, v_t k_t^\top$$

which is exactly the GDN update. Equivalently: starting from $\alpha_t S_{t-1}$, take one gradient-descent step of size β_t on $\frac{1}{2}\|S k_t - v_t\|^2$:

$$S_t = \alpha_t S_{t-1} - \beta_t\big(\alpha_t S_{t-1} k_t - v_t\big) k_t^\top$$

With $\beta_t = 1$ it falls back to the exact solution of Round 29. Neither α_t nor β_t is constant: both are computed from the current token by a small network and change every step. This also answers Round 15's "α is hard to tune": let the model decide from the content how much to decay and how much to write at each step.

Comparison with the GDN paper's line of thought

	This derivation	GDN paper
Role of the state	key→value associative memory	same
Decay	decay αS_{t-1} first, then correct	same (gate α_t)
Objective	$\min \|(\alpha S_{t-1} + A)k_t - (v_t k_t^\top) k_t\|$	$\frac{1}{2}\|S k_t - v_t\|^2$ (identical once k is normalized)
How it is solved	non-unique; take $A = x\,k_t^\top$ and solve exactly	one gradient step, step size β_t

	This derivation	GDN paper
Result	$\alpha S_{t-1}\left(I - \frac{k_t k_t^\top}{k_t^\top k_t}\right) + v_t k_t^\top$	$\alpha_t S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$
Relation	equals the paper with $\beta_t = 1$	a “partial write” of this exact solution

4. Numerical check

With random matrices ($d_v = 5,\ d_k = 4$, $\alpha = 0.9,\ \beta = 0.3$):

- The Round 29 solution satisfies $S_t k_t = \left(k_t^\top k_t\right) v_t$: passes (note: not v_t unless k is normalized).
- With normalized k, the interpolation form, the expanded form and the one-gradient-step form agree exactly.
- Directions orthogonal to k_t are only decayed, never rewritten: passes.

5. Back to ARC: why this affects the prefix cache

A GDN layer compresses its whole history into one fixed-size S_t, and S_t can only be obtained by updating step by step from the start; there is no “S at token N” to cut out on demand. So vLLM can store snapshots of S only at a few positions for these layers: if the conversation prefix changes by even one character before the snapshot, the snapshot is invalid and everything is recomputed. That is why the agent’s conversation was changed to “append only, clean up old material in one batch”.

6. Collaboration review

The grader made several kinds of mistakes in this derivation, recorded here to avoid them next time:

1. **Two out-of-bounds hints**: Round 8 added “what recurses is S_t”; Round 17’s table added the $q = k_j$ row, walking the key step for the deriver.
2. **Grading the literal words, not the meaning**: Round 10’s “attention score”, where Bo clearly meant S_t; Round 25, where a typo ($S_t q_t$) was graded as the main issue — Bo called it nitpicking.
3. **Grading backwards from the final formula**: Round 14’s α^n and Round 22’s “scale it up” were consistent within Bo’s own logic but were graded does-not-hold.
4. **Grading a different object**: in Rounds 13 and 19, about q, one side meant “what is stored in S” and the other “the final expression of o”, costing several exchanges.

The grading format agreed as a result: **first grade “correct within the deriver’s own logic”, then “close to or far from the target”; report only the location, never the direction; write nothing the deriver did not ask for.**

Appendix A: the earliest linear attention is the kernel trick

Before GDN, linear attention was computed directly. The earliest form is nothing more than the kernel trick from SVMs: replace the similarity $\exp(q^\top k)$ with a separable kernel

$$\mathrm{sim}(q,k) = \phi(q)^\top \phi(k), \qquad \phi(x) = \mathrm{elu}(x) + 1 > 0$$

so that

$$o_t = \frac{\left(\sum_{i \le t} v_i\, \phi(k_i)^\top\right) \phi(q_t)}{\phi(q_t)^\top \sum_{i \le t} \phi(k_i)}, \qquad S_t = S_{t-1} + v_t\, \phi(k_t)^\top, \quad z_t = z_{t-1} + \phi(k_t)$$

- ϕ is a positive feature map, so every weight $\phi(q)^\top \phi(k_i)$ is positive;
- the denominator $\phi(q_t)^\top z_t$ makes the weights sum to 1, imitating softmax's normalization.

The states S_t and z_t only accumulate — no decay, no overwrite. The two problems raised in Round 1 (only grows; cannot overwrite old memory) are both still present here, and they are exactly what GDN fixes.